

Fantasy Baseball System Process Doc

- Why This Exists
- How the Ecosystem Works
- What It Produces
- How the Automation Works
- How the Intelligence Reports Work
- Key Files
- Directory Layout
- How to Run Operations
- How to Modify
- Known Quirks and Edge Cases
- If You Are Setting This Up From Scratch
- History

Why This Exists

Before this system existed, managing a Yahoo Fantasy Baseball league meant checking multiple websites manually: Yahoo for rosters and matchups, FanGraphs for projections, Statcast for expected stats, podcast apps for expert takes, and spreadsheets to tie it all together. Roster decisions were gut-feel, trade evaluations were arguments, and waiver adds were whoever you happened to hear about.

This system replaces all of that with a single web app and automated pipeline. It pulls data from Yahoo, FanGraphs, Statcast, and the MLB Stats API, blends projections from multiple systems (Steamer, ZiPS, ATC), calculates fantasy points using the league's specific H2H Points scoring, and surfaces actionable recommendations for lineup optimization, trades, and waivers. On top of that, a daily content pipeline ingests expert blogs and podcast transcripts, then feeds them to Claude to produce personalized intelligence reports tied to the league's actual roster data.

The result: one dashboard at <https://fantasy-baseball-br.fly.dev/> with everything needed to run a competitive fantasy baseball team, plus daily AI-generated analysis delivered to Obsidian for mobile reading.

The league is "Galactic Empire," a 10-team H2H Points keeper league on Yahoo Fantasy Sports.

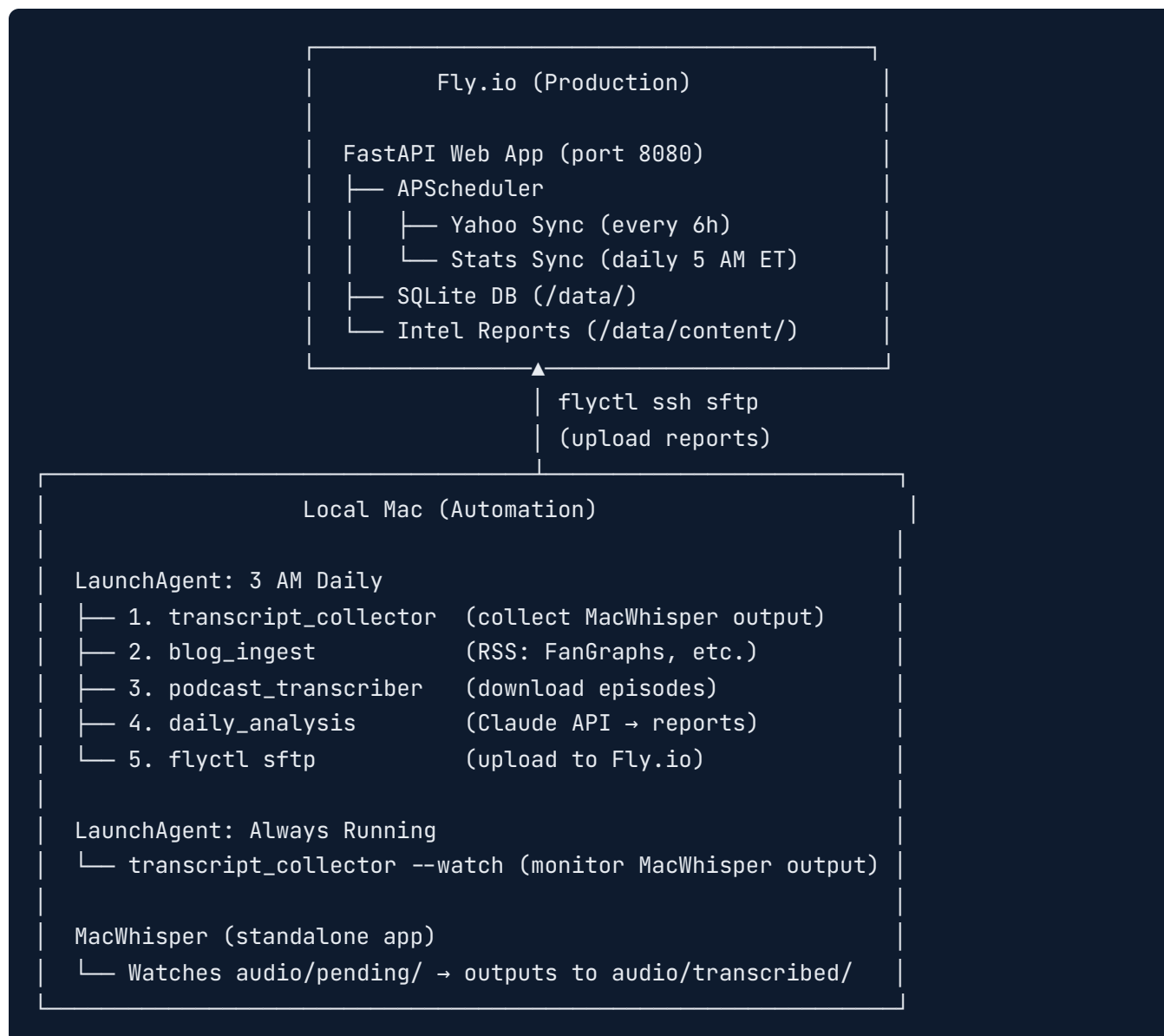
How the Ecosystem Works

There are three subsystems that work together. Understanding how they connect is essential before diving into any one piece.

Subsystem 1: The Web App + Data Pipelines runs on [Fly.io](#) as a FastAPI server. Two scheduled jobs keep data fresh: a Yahoo sync every 6 hours pulls rosters, standings, and transactions; a stats sync at 5 AM ET daily pulls FanGraphs batting/pitching stats, Statcast expected values, and consensus projections. All data lands in a SQLite database on a persistent volume. The web UI serves roster optimization, trade analysis, waiver recommendations, matchup projections, and an interactive stats dashboard.

Subsystem 2: The Content Ingestion Pipeline runs locally on macOS via a LaunchAgent at 3 AM daily. It fetches blog articles from RSS feeds (FanGraphs, Pitcher List, RotoWire), downloads podcast episodes (CBS, FantasyPros, Locked On, In This League) into a MacWhisper watch folder for transcription, and collects finished transcripts. All content is saved as markdown files with YAML frontmatter.

Subsystem 3: The AI Intelligence Reports run as the final step of the daily pipeline. A Python script (`daily_analysis.py`) loads all recent content plus league data from SQLite, sends a single comprehensive prompt to Claude Opus, and splits the response into individual report sections. Reports are saved locally (where Obsidian watches the folder) and uploaded to the [Fly.io](#) volume (where the web app's Intel tab serves them).



What It Produces

Intelligence Reports

The primary output of the content pipeline. Saved as dated markdown files in `data/content/analysis/`.

Daily reports (Mon-Fri, excluding Saturday) contain 4 sections:

- My Roster Intel
- Injury Watch
- Around the League
- Action Items

Monday reports add a recap section:

- Last Week's Recap (standings table, matchup result, player performance)
- Plus all 4 daily sections

Weekly reports (Saturday) are comprehensive with all 11 sections:

- Last Week's Recap
- My Roster Intel
- Injury Watch
- Matchup Preview
- Waiver Targets
- Trade Signals
- Projection Watch
- Around the League
- Cardinals Corner
- Sibling Rivalry
- Action Items

Each report file has YAML frontmatter with title, type, date, generation timestamp, model used, token counts, and content date range. Player names are hyperlinked to FanGraphs profiles. Source citations link to the original article or podcast.

File naming: YYYY-MM-DD_daily-intel.md or YYYY-MM-DD_weekly-intel.md for the combined report, plus individual section files like YYYY-MM-DD_roster-intel.md .

Web App Outputs

- **Optimized Lineups:** PuLP integer linear programming solver maximizes projected fantasy points across roster slots, respecting position eligibility. Output shows each player's optimal slot and projected points, plus total improvement over the current lineup.
- **Trade Values:** Z-score based surplus value over replacement level. Each player gets a positional rank and total value factoring projected points, positional scarcity, and category impact.
- **Waiver Recommendations:** Composite score (0-100) blending projected points (35%), recent performance trend (25%), positional need (15%), league scoring fit (15%), and schedule volume (10%). Includes buy-low signals from Statcast xwOBA deltas.
- **Matchup Projections:** H2H weekly projections with category breakdowns, park factors, platoon splits, and opposing pitcher adjustments.
- **Consensus Projections:** Blended ROS projections from Steamer, ZiPS, and ATC, with Statcast buy/sell signals overlaid.

Content Archive

- `data/content/blogs/*.md` : Blog articles with frontmatter (title, source, date, author, URL)
- `data/content/transcripts/*.md` : Podcast transcripts with frontmatter
- `data/content/manifest.json` : Index of all ingested content (prevents duplicates)

How the Automation Works

Three independent automation systems keep the platform running.

Server-Side: *APScheduler* (*Fly.io*)

Configured in `app/main.py` during the FastAPI lifespan startup.

Yahoo League Sync (`_job_yahoo_sync`)

- Schedule: Every 6 hours (`CronTrigger(hour="*/6")`)
- Job ID: `yahoo_sync`
- Calls: `app.etl.pipeline.run_pipeline()`
- Extracts: stat categories, standings, all team rosters, recent transactions
- Transforms: flattens players, deduplicates by `yahoo_id`, infers stat types
- Loads: upserts players, rosters, standings, stats to SQLite
- Post-load: updates weekly matchup actuals and league-wide snapshots
- Cooldown: 1-minute minimum between syncs (prevents hammering on manual trigger + scheduled overlap)
- Retry: 3 attempts with exponential backoff (2s, 4s, 6s)

Stats Sync (`_job_stats_sync`)

- Schedule: Daily at 5:00 AM Eastern (`CronTrigger(hour=5, timezone="US/Eastern")`)
- Job ID: `stats_sync`
- Calls: `app.etl.pipeline.run_stats_pipeline()`
- Steps (in order):
 1. Seed player ID crosswalk (FanGraphs ID to MLBAM ID mapping)
 2. Fetch FanGraphs batting stats (full season)
 3. Fetch FanGraphs pitching stats (full season)
 4. Fetch Statcast batting summaries (xBA, xSLG, xwOBA, barrel%, exit velo)
 5. Fetch Statcast pitching summaries
 6. Fetch sprint speed data
 7. Update player ages from MLB Stats API
 8. Fetch consensus ROS projections (Steamer + ZiPS + ATC blend)
 9. Fetch Steamer ROS projections (individual, for comparison)
 10. Calculate fantasy points for all players using league scoring weights

Client-Side: LaunchAgents (macOS)

Two LaunchAgent plists live in `~/Library/LaunchAgents/`.

Daily Content Ingest (`com.fantasybaseball.content-ingest`)

- Plist: `~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist`
- Schedule: Daily at 3:00 AM (`StartCalendarInterval: Hour=3, Minute=0`)
- If the Mac is asleep at 3 AM, launchd runs it when the machine wakes up
- Script: `scripts/daily_content_ingest.sh`
- Steps:
 1. Collect MacWhisper transcripts from previous runs
 2. Fetch blog articles via RSS (last 2 days, max 10 articles)
 3. Download new podcast episodes (last 2 days, max 5 episodes)
 4. Generate daily analysis report via Claude API
 5. Upload today's reports to [Fly.io](https://fly.io) volume via `flyctl ssh sftp`
- Each step uses `|| { echo "WARNING: ... failed" }` so one failure does not block the rest
- Log: `data/content/logs/ingest_YYYY-MM-DD.log` (script's own log via `tee`)
- LaunchAgent logs: `data/content/logs/launchd_stdout.log` and `launchd_stderr.log`

Transcript Watcher (`com.fantasybaseball.transcript-watcher`)

- Plist: `~/Library/LaunchAgents/com.fantasybaseball.transcript-watcher.plist`
- Mode: `RunAtLoad=true` , `KeepAlive=true` (starts on login, restarts on exit)
- Command: `uv run python -m scripts.transcript_collector --watch`
- Monitors `data/content/audio/transcribed/` for new MacWhisper output
- When a `.txt` file appears, matches it to the JSON metadata sidecar, wraps it in markdown with frontmatter, and saves to `data/content/transcripts/`
- Log: `data/content/logs/watcher_stdout.log` and `watcher_stderr.log`

Managing LaunchAgents

Load or unload the agents:

```
# Load (start)
launchctl load ~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist
launchctl load ~/Library/LaunchAgents/com.fantasybaseball.transcript-watcher.plist

# Unload (stop)
launchctl unload ~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist
launchctl unload ~/Library/LaunchAgents/com.fantasybaseball.transcript-
watcher.plist

# Check status
launchctl list | grep fantasybaseball

# Trigger manually (runs immediately, independent of schedule)
launchctl kickstart gui/$(id -u)/com.fantasybaseball.content-ingest
```

How the Intelligence Reports Work

The intelligence report system (`scripts/daily_analysis.py`) is the most complex piece of the pipeline. Here is how it works end to end.

Step 1: Determine Report Type

The script auto-selects the report type based on day of week:

- **Saturday** (weekday 5): Full weekly report with all 11 sections, 16,000 max output tokens
- **Monday** (weekday 0): Daily report with last-week recap + 4 daily sections, 8,000 max tokens
- **Tue-Fri**: Lightweight daily briefing with 4 sections, 8,000 max tokens

The `--mode daily|weekly` flag overrides auto-detection. The `--force` flag regenerates even if today's report already exists.

Step 2: Load Content

The script loads all `.md` files from `data/content/blogs/` and `data/content/transcripts/` , parses YAML frontmatter for metadata, and filters by publication date.

- **Weekly reports**: Load all available content (full week review)
- **Daily reports**: Load only content published since the last report's `generated_at` timestamp
- **First run or --force**: Load everything

Full text is sent to Claude with no truncation. Each content item includes title, source name, date, author, and the complete article or transcript body.

Step 3: Load League Context

Reads directly from the SQLite database (not through the async FastAPI services, since this script runs standalone). Queries:

- My full roster with projections, surplus values, and positional ranks
- Ithilien's roster (the brother's team, matched by team name containing "ithilien")
- This week's H2H opponent name and projected points from `weekly_matchup_snapshots`
- The opponent's full roster
- League standings (all 10 teams)
- Top 50 free agents ranked by projected ROS points
- Player name to FanGraphs URL mapping (for hyperlinks in the output)
- Last week's matchup result with per-player stat breakdowns (for Monday recaps)
- Weekly game counts per MLB team from the MLB Stats API

Step 4: Load Previous Sentiments

Reads the most recent `*_roster-intel.md` file (not from today) and parses the sentiment tags (BULLISH, BEARISH, NEUTRAL, NOT MENTIONED) from each player's table. This enables trend tracking in the new report (e.g., "NEUTRAL to BULLISH").

Step 5: Build the Prompt

A single API call structure:

- **System prompt:** Sets the persona as a professional fantasy baseball analyst. Includes formatting rules (`##` headers, **bold** sparingly, *italics* for sources) and content rules (only recommend START/SIT for owned players, use full player names, Cardinals fan context, Ithilien rivalry context).
- **User message:** Assembled from:
 1. Section-specific instructions (each section has a detailed template in `SECTION_INSTRUCTIONS` dict)
 2. League data block (roster, standings, opponent, free agents, game counts, scoring rules)
 3. Previous sentiments block
 4. Full expert content

Step 6: Call Claude API

Model: `claude-opus-4-6`. Single call with retry logic for rate limits (up to 5 retries with increasing wait: 60s, 120s, 180s, 240s, 300s).

Estimated cost per call: ~\$0.15-0.25 for daily, ~\$0.50-1.00 for weekly (at Opus pricing of \$5/M input, \$25/M output).

Step 7: Post-Process and Write

1. **Split into sections:** Regex splits the combined response on `##` headers, matching each to a known section slug
2. **Linkify players:** Two-pass replacement. First pass links all player names in `###` headers to FanGraphs URLs. Second pass links the first body occurrence of each remaining name. Longest names are processed first to avoid partial matches.
3. **Linkify sources:** Matches italicized source citations (e.g., `*Pitcher List, Mar 23*`) to original article URLs from the content items
4. **Write combined report:** Full markdown file with frontmatter, table of contents directives, all linked sections, and a Sources Analyzed appendix
5. **Write individual sections:** Each section as a separate file with its own frontmatter referencing the parent report

Why This Design

Single API call instead of per-section calls: Cheaper (one prompt overhead), more coherent (Claude sees all data when writing each section), and easier to manage rate limits.

Full content, no truncation: Claude Opus handles large contexts well. Truncation risks losing the one mention of a rostered player buried in a podcast transcript. The cost difference is negligible.

Per-section linkification: Each section gets its own “first occurrence” pass so that a player mentioned in Roster Intel and again in Trade Signals gets linked in both places.

Standalone SQLite reads (not async): The script runs outside the FastAPI process, so it uses synchronous `sqlite3` directly rather than importing the async SQLAlchemy session.

Key Files

Scripts

File	Location	Purpose
<code>daily_content_ingest.sh</code>	<code>scripts/</code>	Master daily automation wrapper (3 AM via launchd)
<code>daily_analysis.py</code>	<code>scripts/</code>	Claude-powered intelligence report generator
<code>blog_ingest.py</code>	<code>scripts/</code>	RSS feed fetcher (FanGraphs, Pitcher List, RotoWire)
<code>podcast_transcriber.py</code>	<code>scripts/</code>	Podcast episode downloader to MacWhisper watch folder
<code>transcript_collector.py</code>	<code>scripts/</code>	Collects MacWhisper output, wraps in markdown frontmatter
<code>capture_yahoo_token.py</code>	<code>scripts/</code>	Yahoo OAuth token capture for headless deployment
<code>data_pipeline.py</code>	<code>scripts/</code>	Historical data download (2015-2025) for backtesting
<code>backtest_harness.py</code>	<code>scripts/</code>	Walk-forward projection testing framework
<code>optimize_parameters.py</code>	<code>scripts/</code>	scipy parameter tuning for waiver/projection weights

App Core

File	Location	Purpose
<code>main.py</code>	<code>app/</code>	FastAPI app entry point, middleware, APScheduler jobs
<code>config.py</code>	<code>app/</code>	Pydantic settings (env vars, defaults, season logic)
<code>database.py</code>	<code>app/</code>	SQLAlchemy async engine, session factory, migrations

ETL Pipeline

File	Location	Purpose
<code>pipeline.py</code>	<code>app/etl/</code>	Orchestrates Yahoo sync and stats sync pipelines
<code>extractors.py</code>	<code>app/etl/</code>	Yahoo Fantasy API data extraction via yfpy
<code>transformers.py</code>	<code>app/etl/</code>	Normalizes player data, deduplicates, infers stat types
<code>loaders.py</code>	<code>app/etl/</code>	Upserts to SQLite (players, rosters, stats, standings)

Key Services

File	Location	Purpose
<code>yahoo_service.py</code>	<code>app/services/</code>	Yahoo OAuth management, token refresh, API wrapper
<code>points_service.py</code>	<code>app/services/</code>	Fantasy points calculation using league scoring weights
<code>external_projections.py</code>	<code>app/services/</code>	Fetches Steamer/ZIPS/ATC projections from FanGraphs API
<code>projection_service.py</code>	<code>app/services/</code>	Blends traditional + Statcast stats for buy/sell signals
<code>optimizer_service.py</code>	<code>app/services/</code>	PuLP ILP lineup optimizer
<code>trade_service.py</code>	<code>app/services/</code>	Z-score surplus value trade analyzer
<code>waiver_service.py</code>	<code>app/services/</code>	Composite waiver wire scorer (0-100)
<code>weekly_matchup_service.py</code>	<code>app/services/</code>	H2H matchup projections with park/platoon adjustments
<code>matchup_quality_service.py</code>	<code>app/services/</code>	Park factors, platoon splits, opposing pitcher quality
<code>assistant.py</code>	<code>app/services/</code>	Claude AI chat assistant for in-app queries
<code>id_mapper.py</code>	<code>app/services/</code>	Cross-platform player ID mapping (Yahoo, FanGraphs, MLBAM)

Configuration and Deployment

File	Location	Purpose
<code>fly.toml</code>	project root	Fly.io deployment config (region, volume, env vars)
<code>.env</code>	project root	Local secrets (Yahoo OAuth, Anthropic key, auth password)
<code>pyproject.toml</code>	project root	Dependencies and project metadata (managed by uv)
<code>content-ingest.plist</code>	<code>~/Library/LaunchAgents/</code>	Daily 3 AM content pipeline schedule
<code>transcript-watcher.plist</code>	<code>~/Library/LaunchAgents/</code>	Always-on transcript collection watcher

League Configuration

File	Location	Purpose
league_config.py	app/	Scoring weights, roster slots, replacement levels, team name

Directory Layout

```

fantasy_baseball_br/
├── app/
│   ├── main.py           # FastAPI entry point + APScheduler
│   ├── config.py         # Pydantic settings from .env
│   ├── database.py       # SQLAlchemy async engine + migrations
│   └── league_config.py   # H2H Points scoring weights + roster
├── structure
│   ├── models/           # SQLAlchemy ORM models (14 tables)
│   ├── services/         # Business logic (15 service modules)
│   ├── routes/           # FastAPI route handlers (14 routers)
│   ├── etl/              # Extract-Transform-Load pipeline
│   │   ├── pipeline.py   # Orchestrator (Yahoo + stats pipelines)
│   │   ├── extractors.py # Yahoo API data extraction
│   │   ├── transformers.py # Data normalization
│   │   └── loaders.py    # Database upserts
│   ├── templates/        # Jinja2 HTML templates
│   │   ├── base.html     # Layout with Tailwind + HTMX + Plotly CDN
│   │   ├── partials/     # HTMX partial response fragments
│   │   └── (12 page templates)
│   ├── static/
│   │   ├── css/custom.css
│   │   └── js/           # Charts, comparison, table-sort, tooltips
├── scripts/
│   ├── daily_content_ingest.sh # 3 AM master automation script
│   ├── daily_analysis.py      # Claude-powered report generator
│   ├── blog_ingest.py         # RSS feed fetcher
│   ├── podcast_transcriber.py # Podcast episode downloader
│   ├── transcript_collector.py # MacWhisper transcript processor
│   ├── capture_yahoo_token.py # Yahoo OAuth token bootstrap
│   ├── backtest_harness.py    # Projection backtesting
│   ├── optimize_parameters.py # Parameter tuning
│   ├── data_pipeline.py       # Historical data download
│   └── analysis/              # Analysis scripts (Excel reports)
├── data/
│   ├── content/
│   │   ├── blogs/           # Ingested blog articles (.md)
│   │   ├── transcripts/     # Formatted podcast transcripts (.md)
│   │   ├── analysis/        # Claude-generated intel reports (.md)
│   │   ├── audio/
│   │   │   ├── pending/     # MacWhisper input (.mp3 + .json sidecar)
│   │   │   └── transcribed/ # MacWhisper output (.txt)

```

```

├── logs/                # Pipeline and watcher logs
├── manifest.json        # Content dedup index
├── docs/                # Documentation
├── tests/               # pytest test suite
├── fly.toml             # Fly.io deployment config
├── pyproject.toml       # uv project dependencies
├── .env                 # Local secrets (not committed)
└── fantasy_baseball.db  # Local SQLite database

```

How to Run Operations

Start the Dev Server

```

cd /Users/brianrenshaw/Projects/fantasy_baseball_br
uv run uvicorn app.main:app --reload --port 8000

```

The app starts at `http://localhost:8000`. APScheduler jobs begin running on their schedules immediately.

Run a Manual Yahoo Sync

From the web UI, use the sync button on the dashboard. Or trigger the pipeline directly:

```

uv run python -c "import asyncio; from app.etl.pipeline import run_pipeline;
print(asyncio.run(run_pipeline()))"

```

Run a Manual Stats Sync

```

uv run python -c "import asyncio; from app.etl.pipeline import run_stats_pipeline;
print(asyncio.run(run_stats_pipeline()))"

```

This takes several minutes. It fetches from FanGraphs, Statcast, and MLB Stats API, then recalculates all player points.

Run the Content Pipeline Manually

Run the full daily pipeline:

```

./scripts/daily_content_ingest.sh

```

Or run individual steps:

```
# Collect any waiting MacWhisper transcripts
uv run python -m scripts.transcript_collector

# Fetch blog articles (last 7 days)
uv run python -m scripts.blog_ingest --days 7

# Download podcast episodes (last 3 days, max 5 per feed)
uv run python -m scripts.podcast_transcriber --days 3 --max-episodes 5

# Generate analysis report
uv run python -m scripts.daily_analysis

# Preview the prompt without calling Claude
uv run python -m scripts.daily_analysis --dry-run

# Force regeneration even if today's report exists
uv run python -m scripts.daily_analysis --force

# Generate a weekly report regardless of day
uv run python -m scripts.daily_analysis --mode weekly
```

View Logs

```
# Daily pipeline log (today)
cat data/content/logs/ingest_$(date +%Y-%m-%d).log

# LaunchAgent stdout/stderr
cat data/content/logs/launchd_stdout.log
cat data/content/logs/launchd_stderr.log

# Transcript watcher log
cat data/content/logs/watcher_stdout.log

# Fly.io server logs
flyctl logs --no-tail --app fantasy-baseball-br
```

Deploy to Fly.io

```
flyctl deploy
```

The Dockerfile handles the build. The persistent volume at `/data` survives deployments. After deploying, verify:

```
flyctl logs --no-tail
curl https://fantasy-baseball-br.fly.dev/health
```

Upload Reports to *Fly.io* Manually

```
flyctl ssh sftp shell --app fantasy-baseball-br <<EOF
put data/content/analysis/2026-03-27_daily-intel.md /data/content/analysis/2026-03-
27_daily-intel.md
EOF
```

How to Modify

Add a New RSS Feed

Edit `scripts/blog_ingest.py` . Add an entry to the `FEEDS` dict:

```
FEEDS = {
    "fangraphs": "https://blogs.fangraphs.com/feed/",
    "pitcherlist": "https://pitcherlist.com/feed",
    "rotowire": "https://www.rotowire.com/rss/news.php?sport=MLB",
    "new_source": "https://example.com/feed/rss", # add here
}
```

The script auto-discovers article content from the feed entries. If the new site has unusual HTML structure, you may need to add a custom content extractor.

Add a New Podcast Feed

Edit `scripts/podcast_transcriber.py` . Add an entry to the `FEEDS` dict with the podcast's RSS feed URL.

Add a New Report Section

In `scripts/daily_analysis.py` :

1. Add the section slug and display title to the `SECTIONS` dict
2. Add the slug to the appropriate list (`DAILY_SECTIONS` , `MONDAY_SECTIONS` , or `WEEKLY_SECTIONS`)
3. Add detailed instructions in `SECTION_INSTRUCTIONS` dict (this is the prompt template Claude follows)

Change the Daily Pipeline Schedule

Edit `~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist` . Change the `StartCalendarInterval` :

```
<key>StartCalendarInterval</key>
<dict>
  <key>Hour</key>
  <integer>4</integer>    <!-- Change to 4 AM -->
  <key>Minute</key>
  <integer>30</integer>   <!-- At 4:30 -->
</dict>
```

Then reload:

```
launchctl unload ~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist
launchctl load ~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist
```

Change the Yahoo or Stats Sync Schedule

Edit `app/main.py` in the `lifespan` function. The schedules use APScheduler's `CronTrigger` :

```
# Yahoo sync: change from every 6 hours to every 4 hours
scheduler.add_job(_job_yahoo_sync, CronTrigger(hour="*/4"), ...)

# Stats sync: change from 5 AM ET to 6 AM ET
scheduler.add_job(_job_stats_sync, CronTrigger(hour=6, timezone="US/Eastern"), ...)
```

Redeploy to [Fly.io](https://fly.io) for changes to take effect in production.

Update League Scoring Weights

Edit `app/league_config.py` . The `BATTING_SCORING` and `PITCHING_SCORING` dicts define point values per stat. Every service that calculates fantasy points (`points_service`, `optimizer`, `waivers`, `trades`) reads from this file.

Add a New Data Source

1. Create a new service in `app/services/` following the existing pattern (async functions, executor for blocking calls)
2. Add extraction logic to `app/etl/extractors.py` or as a standalone service
3. Add a database model in `app/models/` if new data needs its own table
4. Wire it into `app/etl/pipeline.py`'s `run_stats_pipeline()` function
5. Add a migration in `app/database.py`'s `init_db()` if adding columns to existing tables

Known Quirks and Edge Cases

Yahoo OAuth tokens expire hourly. The `yahoo_service.py` auto-refreshes tokens on each API call and persists the refreshed token to `/data/yahoo_token.json` on the [Fly.io](#) volume. If the token goes stale (e.g., [Fly.io](#) volume is recreated), you need to re-run `scripts/capture_yahoo_token.py` locally and update the Fly secret.

MacWhisper is a separate desktop app. The podcast pipeline downloads `.mp3` files to `data/content/audio/pending/`, but transcription happens in MacWhisper (not part of this codebase). MacWhisper must be configured to watch the `pending/` folder and output `.txt` files to `transcribed/`. If MacWhisper is not running, transcripts do not appear. The next daily pipeline run picks up any transcripts that accumulated.

SQLite timestamps must be UTC. All `datetime` values stored in the database use `datetime.now(timezone.utc)`. Never use naive `datetime.now()`. This has caused bugs before.

pybaseball and FanGraphs have rate limits. The `external_projections.py` service includes retry logic (3 retries, 5-second delay) and a User-Agent header to avoid bot detection. FanGraphs occasionally changes their API structure, which breaks the pybaseball integration.

The stats pipeline takes 3-5 minutes. It makes many sequential HTTP calls to FanGraphs and Statcast. Do not run it more frequently than once per day.

Content dedup relies on manifest.json. The `blog_ingest.py` script tracks ingested article URLs in `data/content/manifest.json`. If this file is deleted, articles will be re-ingested (but with the same filenames, so no duplicates in the filesystem).

Claude daily analysis skips if today's report exists. If the script runs twice in one day (e.g., launchd catchup after sleep + manual run), the second run exits immediately unless `--force` is passed.

The weekly report day is Saturday (weekday 5). This is hardcoded in `WEEKLY_DAY` in `daily_analysis.py`. Change it there if you want the weekly report on a different day.

Player ID mapping is imperfect. The `id_mapper.py` service maps between Yahoo, FanGraphs, MLBAM, and Baseball Reference IDs. Name matching is fuzzy. International players and players with name changes (e.g., accent marks) sometimes fail to map. The crosswalk seeding step in the stats pipeline handles most cases.

The [Fly.io](#) volume is a single mount point. Database, cache, content, and tokens all live under `/data`. If the volume fills up, everything stops. Current VM spec is 1 GB RAM, shared CPU. The SQLite database is typically 50-100 MB.

If You Are Setting This Up From Scratch

Prerequisites

- macOS (for LaunchAgents and MacWhisper)
- Python 3.11+
- [uv](#) package manager
- [MacWhisper](#) desktop app (for podcast transcription)
- [flyctl](#) CLI (for deployment)
- A Yahoo Developer App with Fantasy Sports API access
- An Anthropic API key

Step 1: Clone and Install Dependencies

```
git clone <repo-url> fantasy_baseball_br
cd fantasy_baseball_br
uv sync
```

Step 2: Create .env File

```
cp .env.example .env # or create manually
```

Required variables:

```
YAHOO_CLIENT_ID=your_client_id
YAHOO_CLIENT_SECRET=your_client_secret
YAHOO_LEAGUE_ID=your_league_id
ANTHROPIC_API_KEY=your_api_key
AUTH_PASSWORD=your_password
```

Step 3: Yahoo OAuth Setup

1. Go to <https://developer.yahoo.com/apps/create/>
2. Create an "Installed Application" with "Fantasy Sports" API permission
3. Save the Client ID and Client Secret to `.env`
4. Start the dev server: `uv run uvicorn app.main:app --reload --port 8000`
5. Visit `http://localhost:8000` and complete the browser-based OAuth flow
6. The token auto-refreshes after initial authorization

For headless ([Fly.io](#)) deployment:

```
uv run python -m scripts.capture_yahoo_token
# Copy the output JSON
flyctl secrets set YAHOO_ACCESS_TOKEN_JSON='<paste json>'
```

Step 4: Initialize the Database

The database auto-creates on first server start (`init_db()` in the lifespan). To populate it:

1. Start the dev server
2. Trigger a Yahoo sync (dashboard sync button or manual pipeline run)
3. Wait for the stats pipeline to run (or trigger manually)

Step 5: Set Up Content Pipeline

Create the content directory structure:

```
mkdir -p data/content/{blogs,transcripts,analysis,logs}
mkdir -p data/content/audio/{pending,transcribed}
```

Configure MacWhisper:

- Watch folder: `<project>/data/content/audio/pending/`
- Output folder: `<project>/data/content/audio/transcribed/`
- Output format: `.txt`

Step 6: Install LaunchAgents

Copy the plists and update paths:

```
cp <wherever>/com.fantasybaseball.content-ingest.plist ~/Library/LaunchAgents/
cp <wherever>/com.fantasybaseball.transcript-watcher.plist ~/Library/LaunchAgents/
```

Edit both plists to update the project directory path if different from

`/Users/brianrenshaw/Projects/fantasy_baseball_br` . Then load:

```
launchctl load ~/Library/LaunchAgents/com.fantasybaseball.content-ingest.plist
launchctl load ~/Library/LaunchAgents/com.fantasybaseball.transcript-watcher.plist
```

Step 7: Deploy to *Fly.io*

```
flyctl launch # first time only
flyctl volumes create data_vol --region ord --size 1
flyctl secrets set YAHOO_CLIENT_ID=... YAHOO_CLIENT_SECRET=... YAHOO_LEAGUE_ID=...
ANTHROPIC_API_KEY=... AUTH_PASSWORD=...
flyctl deploy
```

Step 8: Verify Everything Works

1. Visit `https://fantasy-baseball-br.fly.dev/health` (should return OK)
2. Log in and check the dashboard for roster data
3. Run `./scripts/daily_content_ingest.sh` manually and check for report output in `data/content/analysis/`
4. Check the Intel tab in the web app for uploaded reports

History

Date	Change
2025 Q1	Initial project created: FastAPI app with Yahoo integration, SQLite database
2025 Q1	Added FanGraphs and Statcast data pipelines via pybaseball
2025 Q1	Added PuLP lineup optimizer, trade analyzer, waiver scorer
2025 Q1	Deployed to Fly.io with persistent volume
2025 Q2	Added APScheduler for automated Yahoo (6h) and stats (daily) syncs
2025 Q2	Added Claude AI chat assistant in web app
2025 Q2	Built content pipeline: blog ingest, podcast transcriber, transcript collector
2025 Q2	Added daily_analysis.py for Claude-powered intelligence reports
2025 Q2	Set up LaunchAgents for 3 AM automation and transcript watching
2025 Q2	Added Intel tab to web app for serving reports
2025 Q3	Renamed to "Lankford Legends," redesigned mobile dashboard
2025 Q3	Added H2H Points league scoring throughout (SV=7, HLD=4, OUT=1.5)
2025 Q3	Added weekly matchup snapshots and league-wide scoreboard tracking
2025 Q3	Added projection accuracy tracking (Yahoo vs app projections)
2025 Q4	Backtesting framework and parameter optimization added
2026 Q1	Report system upgraded: per-section linkification, sentiment trend tracking
2026 Q1	Added Monday recap sections with per-player stat breakdowns